

STRIDE

Supply chain Tracking, Routing, Inventory & Distribution Engine —
Complete System Design, Architecture & Implementation Specification

Abhinand Baiju Smitha

Final Year Project — Presidency University, Department of Computer
Science and Engineering

Contents

1	Executive Summary	4
2	Problem Statement & Motivation	5
2.1	The Real-World Problem	5
2.2	Why This Is a Good Final-Year Project	5
2.3	Non-Goals	5
3	Originality & Differentiation from Existing Tutorials	7
4	Requirements Specification	8
4.1	Functional Requirements	8
4.2	Non-Functional Requirements	9
5	System Architecture	10
5.1	Layered Architecture Overview	10
5.2	Package Structure	10
5.3	Why the Routing Service Is Isolated (Design Rationale)	11
6	Technology Stack & Justification	12
7	Domain Model	14
7.1	Entity-Relationship Diagram (textual)	14
7.2	Entity Specifications	14
7.2.1	Warehouse	14
7.2.2	Product	14
7.2.3	StockItem	15
7.2.4	Customer	15
7.2.5	Order	15
7.2.6	OrderLine	15
7.2.7	OrderAllocation	16
7.2.8	User	16

8	Order Lifecycle & State Machine	17
8.1	Order-Level States	17
8.2	Allocation-Level States (per OrderAllocation row)	17
8.3	Transition Table	17
8.4	Why a State Pattern, Not an if/else Chain	17
9	REST API Design	19
9.1	Auth	19
9.2	Warehouses	19
9.3	Products	19
9.4	Stock	19
9.5	Orders (the core flow)	20
9.5.1	Example — POST /orders request	20
9.5.2	Example — POST /orders response	20
9.6	Error Response Shape (consistent across all endpoints)	21
10	Core Algorithm Deep Dive: Fulfillment Routing	22
10.1	Problem Formulation	22
10.2	Why Not Just Solve the ILP?	22
10.3	Phase 1 — Candidate Filtering	23
10.4	Phase 2 — Per-Line Greedy Scoring	23
10.5	Phase 3 — Basket-Level Consolidation (Bitmask DP)	24
10.6	Complexity Summary	24
10.7	Reference Implementation (Java)	25
10.8	Consolidation Planner (Bitmask DP)	27
11	Object-Oriented Design & Design Patterns	29
11.1	Strategy Pattern — RoutingStrategy	29
11.2	State Pattern — OrderState	29
11.3	Builder Pattern — AllocationPlan construction	29
11.4	Single Responsibility Principle — the three-way split	29
11.5	Composition Over Inheritance	30
11.6	Repository Pattern (via Spring Data JPA)	30
12	Concurrency & Data Consistency	31
12.1	The Race Condition	31
12.2	Why Optimistic Locking, Not Pessimistic	31
12.3	Retry Scope — the Detail Most Implementations Get Wrong	32
12.4	Reserved vs. Available Quantity	32
12.5	Isolation Level	33
13	Security Design	34
13.1	Authentication Flow	34
13.2	Authorization — Method-Level, Not Just URL-Level	34
13.3	Password & Secret Handling	34
14	Database Schema (Flyway Migration)	35
15	DSA Concepts Applied — Summary Table	38

16 Testing Strategy	40
16.1 Philosophy	40
16.2 Unit Tests — Pure Algorithm (no Spring context)	40
16.3 Integration Tests — Testcontainers	41
16.4 Concurrency Test — the Most Important Test in the Project	42
16.5 Coverage Targets	43
17 Error Handling & Validation	44
17.1 Global Exception Handler	44
17.2 Bean Validation on Input DTOs	44
18 Scheduled Jobs & Twilio Reorder Alerts	46
19 DevOps & Deployment	47
19.1 docker-compose.yml	47
19.2 Dockerfile (multi-stage build)	47
19.3 Environment Variables (.env.example)	48
20 Optional Demo Dashboard	49
21 Three-Week Build Plan (Day by Day)	50
21.1 Week 1 — Foundations	50
21.2 Week 2 — The Core Differentiator	50
21.3 Week 3 — Polish, Proof, Presentation	50
22 Evaluation Criteria Mapping	52
23 Viva / Evaluation Q&A Preparation	53
24 Future Work & Extensions	55
25 Appendix	56
25.1 Full Entity List	56
25.2 Glossary	56
25.3 Maven Dependencies (pom.xml excerpt)	56
25.4 Closing Note	56

1 Executive Summary

STRIDE is a backend-first supply chain fulfillment platform built on **Java 17 and Spring Boot 3**, designed to solve a problem that most academic inventory-management projects deliberately avoid: **deciding how an order should actually be fulfilled when stock is spread across multiple warehouses.**

Where a typical CRUD-based inventory project stops at “check quantity, subtract quantity,” STRIDE implements an **explainable, tunable fulfillment routing engine** that:

- Scores candidate warehouses for each order line using a weighted combination of distance, cost, and stock shortfall.
- Splits an order across warehouses only when no single warehouse can serve it economically, using a **bitmask dynamic programming consolidation step** to avoid unnecessary shipment fragmentation.
- Handles **concurrent checkout races** correctly using optimistic locking (@Version) with a retry boundary that re-runs the *entire* routing decision, not just the database write.
- Exposes the whole system through a secured, documented, tested REST API.

The project is deliberately scoped as **one Spring Boot service done deeply**, rather than a sprawling microservice architecture, so that the engineering effort goes into correctness, concurrency, and algorithmic justification rather than infrastructure glue. Every non-trivial design decision in this document is accompanied by an explicit justification of the alternative that was rejected and why — this is intentional, because the strongest signal in a final-year evaluation is not “it works” but “I understood the tradeoff and chose deliberately.”

This document is the complete specification: problem framing, requirements, architecture, data model, algorithm design with complexity analysis, OOP design pattern usage, security design, database schema, testing strategy, DevOps setup, a day-by-day three-week build plan, and a viva preparation section anticipating the questions an evaluator is likely to ask.

2 Problem Statement & Motivation

2.1 The Real-World Problem

Multi-location retailers and distributors — pharmacies, electronics distributors, FMCG chains — routinely face a decision that looks simple but isn't: **a customer places an order, and the system must decide which warehouse (or combination of warehouses) fulfills it.**

Naive systems make this decision with a single rule: “fulfill from the nearest warehouse with enough stock.” This breaks down constantly in practice:

1. **The nearest warehouse doesn't have enough stock.** The naive system either rejects the order or silently backorders it, when a nearby-but-not-nearest warehouse could have filled it completely.
2. **Splitting is sometimes right, sometimes wrong.** Splitting an order across two warehouses can reduce total shipping cost per unit, but it also means two shipments, two packing operations, and a worse customer experience. A good system should split only when the benefit outweighs the fragmentation cost.
3. **Concurrent orders compete for the same stock.** During a flash sale or high-order-volume period, two orders can both “see” the same available unit and both try to claim it. A system without correct concurrency control either oversells (bad) or serializes every request through a single lock (slow).

2.2 Why This Is a Good Final-Year Project

A final-year CSE project is evaluated on more than “does it run.” The strongest projects demonstrate:

- **A real algorithmic decision**, not just data movement (CRUD).
- **Correct handling of concurrency**, which is one of the few topics that reliably separates a strong engineer from someone who followed a tutorial.
- **Deliberate use of OOP and design patterns** to solve actual structural problems, not patterns bolted on for the sake of a checklist.
- **A testing strategy** that proves the hard parts (the algorithm, the concurrency) actually work, not just that the happy path returns HTTP 200.

STRIDE is scoped specifically to hit all four of these, using a domain (supply chain fulfillment routing) that is genuinely underexplored in student projects, most of which stop at basic inventory CRUD.

2.3 Non-Goals

To keep the three-week scope realistic, the following are explicitly **out of scope** for the core deliverable:

- Full route optimization / vehicle routing (this is a warehouse *selection* problem, not a delivery-route problem).
- Machine-learning-based demand forecasting (mentioned only as a “Future Work” extension).

- A production-grade payment gateway integration (a stubbed/mockd payment step is sufficient).
- Multi-tenant SaaS support (single-organization system).

3 Originality & Differentiation from Existing Tutorials

Nearly every publicly available “Spring Boot inventory management” tutorial or student repository implements the same shape of system: Product, Order, Customer entities, basic CRUD, and a single `reduceStock()` method that subtracts a quantity from one row. STRIDE differs in three concrete, checkable ways that should be stated explicitly in the project report’s originality section:

Aspect	Typical tutorial project	STRIDE
Fulfillment decision	Implicit — first/only warehouse is used	Explicit scoring + greedy + DP consolidation algorithm
Concurrency	Usually absent, or a blanket synchronized block	@Version optimistic locking with whole-use-case retry
Splitting orders	Not modeled — one order maps to one shipment	OrderAllocation join entity allows one order line to map to multiple warehouses
Algorithmic justification	None — code is presented without alternatives discussed	Explicit greedy-vs-ILP tradeoff, complexity analysis, and a documented reason for the chosen approach
Testing depth	Manual Postman testing, if any	JUnit unit tests for the pure algorithm + Testcontainers integration tests including a deliberate concurrent-order race test

The routing algorithm, the bitmask consolidation step, and the retry-scope design are original work applied to this specific domain — they are standard *algorithmic techniques* (greedy heuristics, subset DP, optimistic concurrency control) but their combination and application to warehouse fulfillment routing is the project’s own design, not copied from a tutorial. The report should cite this table directly when asked “how is this different from every other inventory project.”

4 Requirements Specification

4.1 Functional Requirements

Note: WH_MANAGER is used throughout this document as a shorthand for the WAREHOUSE_MANAGER role, purely for table-width readability.

FR-1 — Authentication & Authorization

- FR-1.1. Users register/log in and receive a JWT access token and refresh token.
- FR-1.2. Every protected endpoint enforces role-based access: ADMIN, WH_MANAGER, SUPPLIER.
- FR-1.3. A WH_MANAGER may only view/manage stock for warehouses they are assigned to.
- FR-1.4. A SUPPLIER may only view stock and orders relevant to products they supply.

FR-2 — Master Data Management

- FR-2.1. ADMIN can create, update, deactivate Warehouse records (name, address, lat/lng, cost factor).
- FR-2.2. ADMIN and SUPPLIER can create/update Product records (SKU, name, category, unit weight).
- FR-2.3. WH_MANAGER can create/update StockItem records (product + warehouse + quantity + batch/expiry if applicable).

FR-3 — Order Placement & Fulfillment Routing

- FR-3.1. A customer-facing endpoint accepts an order (customer + list of product/quantity lines).
- FR-3.2. The system computes a fulfillment plan per order using the routing algorithm (Section 10).
- FR-3.3. If full coverage is impossible, the uncovered portion is marked BACKORDERED and the order status becomes PARTIALLY_ALLOCATED.
- FR-3.4. The order's computed allocation (which warehouse fulfills how much of which line) is persisted as OrderAllocation records.
- FR-3.5. Stock is decremented atomically and correctly under concurrent order placement (no overselling).

FR-4 — Order Lifecycle Management

- FR-4.1. Orders progress through a defined state machine (Section 8.5) with illegal transitions rejected.
- FR-4.2. WH_MANAGER can transition an order's allocation from ALLOCATED → PICKED → SHIPPED.
- FR-4.3. ADMIN can cancel an order in any pre-SHIPPED state, releasing reserved stock back to available inventory.

FR-5 — Reorder Alerts

- FR-5.1. A scheduled job runs daily, checking each StockItem against its product's reorder threshold.

- FR-5.2. Items below threshold trigger an SMS alert (via Twilio) to the relevant `WH_MANAGER`.

FR-6 — Reporting & Query

- FR-6.1. Paginated, filterable listing of orders (by status, date range, customer).
- FR-6.2. Paginated, filterable listing of stock (by warehouse, product, below-threshold flag).
- FR-6.3. An endpoint returning the full allocation breakdown for a given order (for the demo dashboard / map view).

4.2 Non-Functional Requirements

NFR-1 — Correctness under concurrency. No two concurrent order placements may oversell the same `StockItem`. Verified with an automated concurrency test (Section 16.4).

NFR-2 — Explainability. The routing algorithm must be a pure function that can be unit tested without a database or Spring context, and its output must include the score breakdown used for each allocation decision (for auditability and for the demo).

NFR-3 — Reasonable performance bound on routing. The bitmask DP consolidation step is only run over a *pre-filtered* candidate set (Section 10.5), capped at ~15 warehouses per order, to keep the 2^k term bounded regardless of total warehouse count in the system.

NFR-4 — Security baseline. Passwords hashed with BCrypt; JWT signed with a rotated secret; role checks enforced at the method level (`@PreAuthorize`), not only at the UI.

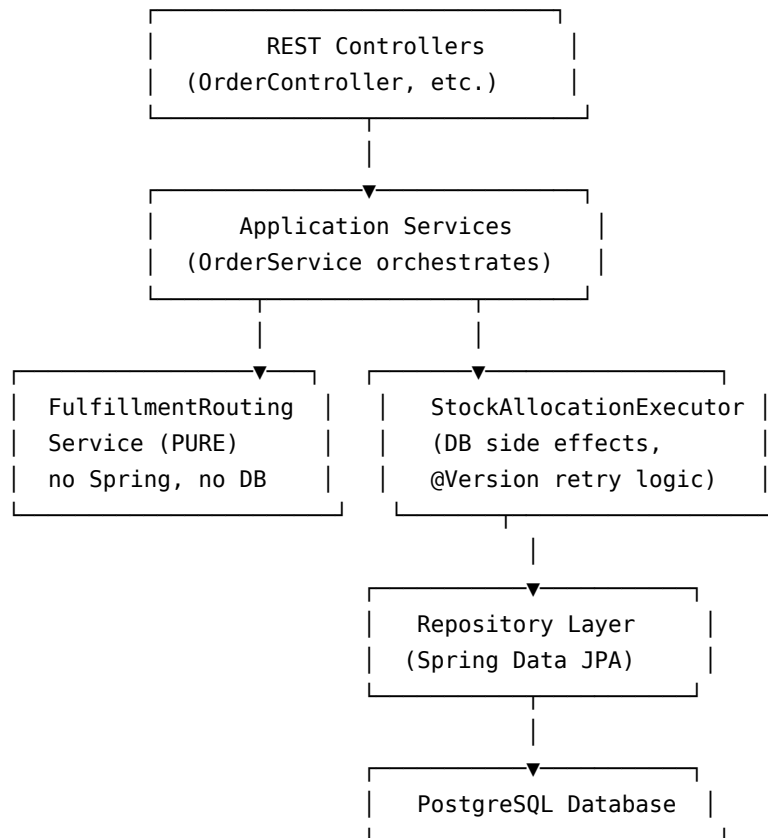
NFR-5 — Testability. Minimum coverage target: 100% of the routing/consolidation algorithm branches, and integration coverage of the full order-placement happy path plus the backorder and race-condition edge cases.

NFR-6 — Deployability. The full system (app + Postgres) must start with a single docker compose up.

5 System Architecture

5.1 Layered Architecture Overview

STRIDE follows a classic layered architecture, with one deliberate deviation: the fulfillment routing logic is isolated into its **own layer**, separate from the standard service layer, because it must remain framework-free and side-effect-free (Section 5.3 explains why).



Cross-cutting concerns sit alongside this stack:

- **Security filter chain** (JWT validation) intercepts before the controller layer.
- **Global exception handler** (@ControllerAdvice) wraps the controller layer.
- **Scheduler** (@Scheduled) runs independently, calling into the service layer for the reorder-check job.
- **Twilio client** is invoked from the scheduler/notification service, not from the core order flow (keeps the critical path free of a third-party network call).

5.2 Package Structure

```
com.smartroute
├─ config/           # SecurityConfig, OpenApiConfig, RetryConfig
├─ security/         # JwtFilter, JwtService, UserDetailsServiceImpl
└─ domain/
```

```

|   ├── entity/                # JPA entities
|   ├── enums/                 # OrderStatus, Role, AllocationStatus
|   └── state/                  # OrderState pattern implementation
├── repository/                 # Spring Data JPA repositories
├── routing/
|   ├── FulfillmentRoutingService.java # pure algorithm
|   ├── RoutingStrategy.java          # Strategy interface
|   ├── DistanceCostStrategy.java     # default implementation
|   ├── ConsolidationPlanner.java     # bitmask DP
|   └── model/                        # AllocationPlan, WarehouseScore (DTOs internal to routing)
├── service/
|   ├── OrderService.java             # orchestration, @Transactional, @Retryable
|   ├── StockAllocationExecutor.java   # DB writes for allocation + stock decrement
|   ├── WarehouseService.java, ProductService.java, StockService.java
|   └── ReorderAlertService.java      # scheduled job logic
├── notification/
|   └── TwilioSmsClient.java
├── web/
|   ├── controller/                # REST controllers
|   ├── dto/                       # Request/response DTOs
|   └── exception/                  # GlobalExceptionHandler, custom exceptions
└── STRIDEApplication.java

```

5.3 Why the Routing Service Is Isolated (Design Rationale)

The `FulfillmentRoutingService` takes an `OrderRequest` and a `List<StockSnapshot>` (a plain Java record, not a JPA entity) and returns an `AllocationPlan` (also a plain record). It has **no Spring annotations, no repository injections, no database access**.

This is deliberate, for three reasons:

1. **Testability.** The hardest, most bug-prone part of the system — the algorithm — can be tested with plain JUnit in milliseconds, with no Spring context startup and no database. This is the single biggest lever for confidence in correctness.
2. **Correctness under retry.** Because the routing decision depends on a stock snapshot that can go stale between “read” and “write” under concurrency, the *entire* routing decision must be re-computed on retry (Section 12.3), not just the database write. Keeping the algorithm pure and side-effect-free makes it safe to call it repeatedly without corrupting state.
3. **Strategy substitutability.** Because `FulfillmentRoutingService` depends on a `RoutingStrategy` interface, a future extension (e.g., an ILP-based strategy, or a machine-learned cost model) can be swapped in without touching the orchestration or persistence code.

6 Technology Stack & Justification

Layer	Choice	Justification
Language	Java 17	LTS release; records and pattern matching used for the routing DTOs (Section 10)
Framework	Spring Boot 3.2	Industry-standard; Spring Security, Spring Data JPA, Spring Retry integrate cleanly
Database	PostgreSQL 15	Row-level locking support, <code>SERIALIZABLE/REPEATABLE READ</code> isolation options relevant to Section 12; JSONB available if needed for audit logs
Migrations	Flyway	Versioned schema history — avoids <code>ddl-auto=update</code> , which is explicitly a red flag in evaluation since it hides schema-design decisions
Security	Spring Security + JJWT	Stateless JWT auth; standard, well-documented, testable
Testing	JUnit 5, Mockito, Testcontainers, REST Assured	Testcontainers gives integration tests against a <i>real</i> Postgres instance, not H2 — catches real SQL/locking behavior differences
Retry	Spring Retry (@Retryable)	Declarative retry on <code>ObjectOptimisticLockingFailureException</code> , scoped correctly (Section 12.3)
API docs	springdoc-openapi	Auto-generated, always in sync with code, free
Notifications	Twilio Java SDK	Reorder alerts via SMS — small, well-justified integration point
Containerization	Docker + Docker Compose	Single-command startup; matches NFR-6

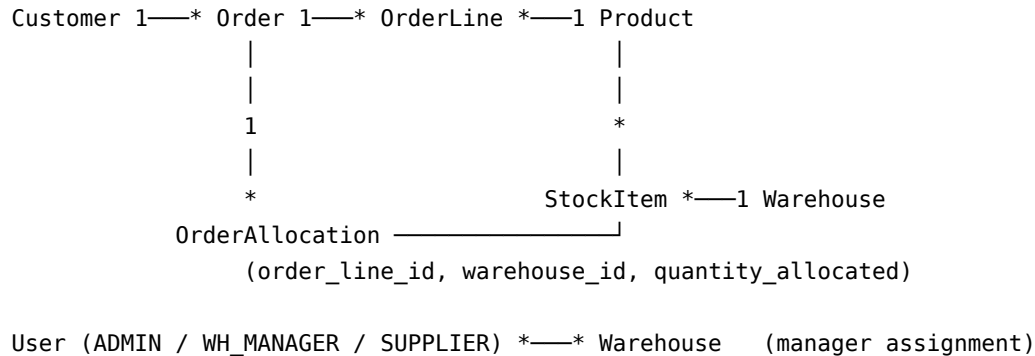
Layer	Choice	Justification
Build	Maven	More common in enterprise Java shops than Gradle for this kind of service; also simpler for a single-module project

Rejected alternatives worth naming in the report (this signals deliberate choice, not default-picking):

- **MongoDB** was considered for order storage but rejected — the domain is inherently relational (orders, lines, allocations, warehouses all reference each other with strong consistency requirements), and MongoDB’s eventual-consistency defaults work against the concurrency guarantees this project is built to demonstrate.
- **synchronized blocks / pessimistic locking (SELECT ... FOR UPDATE)** were considered for the stock-decrement step but rejected in favor of optimistic locking, because pessimistic locks serialize all writers to a given warehouse’s stock row, which does not scale under high order volume; optimistic locking only pays a cost (a retry) when an actual conflict occurs.
- **A full ILP solver (e.g., OR-Tools)** was considered for the routing decision but rejected in favor of a greedy heuristic + bounded bitmask DP, because an ILP solve is not guaranteed to return within a bounded time on every request, which is unacceptable for a synchronous “place order” API call. This tradeoff is documented explicitly in Section 10.6.

7 Domain Model

7.1 Entity-Relationship Diagram (textual)



7.2 Entity Specifications

7.2.1 Warehouse

Field	Type	Notes
id	UUID	PK
name	String	unique, not null
address	String	
latitude	double	used in distance scoring
longitude	double	used in distance scoring
costFactor	double	relative operating/shipping cost multiplier, e.g. 1.0 = baseline
active	boolean	inactive warehouses excluded from routing candidates
createdAt / updatedAt	Instant	audit timestamps

7.2.2 Product

Field	Type	Notes
id	UUID	PK
sku	String	unique
name	String	
category	String	
unitWeightKg	double	optional, for future shipping-cost refinement
reorderThreshold	int	per-product default; can be overridden per StockItem

7.2.3 StockItem

Field	Type	Notes
id	UUID	PK
warehouse	Warehouse	FK, not null
product	Product	FK, not null
quantity	int	current available quantity
reservedQuantity	int	quantity allocated to unshipped orders (see Section 12)
batchExpiry	LocalDate	nullable — populated for perishable-goods extension
version	long	@Version — optimistic locking field, the single most important column in this schema

Design note: quantity tracks *physically present* stock; reservedQuantity tracks stock already promised to open orders. Available-to-promise = quantity - reservedQuantity. This separation is what allows an order to be *allocated* (reserved) before it is *shipped* (physically deducted), which matches how real warehouse systems behave and gives you a clean point to release stock on cancellation.

7.2.4 Customer

Field	Type	Notes
id	UUID	PK
name, email, phone	String	
address	String	
latitude, longitude	double	used for distance scoring in routing

7.2.5 Order

Field	Type	Notes
id	UUID	PK
customer	Customer	FK
status	OrderStatus (enum)	see state machine, Section 8.5
createdAt	Instant	
orderLines	List<OrderLine>	one-to-many, cascade persist

7.2.6 OrderLine

Field	Type	Notes
id	UUID	PK
order	Order	FK
product	Product	FK
quantityRequested	int	
allocations	List<OrderAllocation>	one-to-many; empty until routing runs

7.2.7 OrderAllocation

This is the entity that makes order splitting possible — the direct output of the routing algorithm.

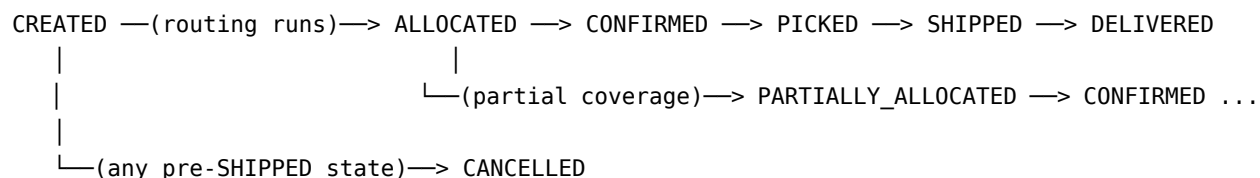
Field	Type	Notes
id	UUID	PK
orderLine	OrderLine	FK
warehouse	Warehouse	FK
quantityAllocated	int	
scoreBreakdown	JSONB / String	serialized score components used for this decision — supports NFR-2 (explainability)
status	AllocationStatus (enum)	ALLOCATED, PICKED, SHIPPED

7.2.8 User

Field	Type	Notes
id	UUID	PK
email	String	unique, used as login
passwordHash	String	BCrypt
role	Role (enum)	ADMIN, WH_MANAGER, SUPPLIER
assignedWarehouses	List<Warehouse>	many-to-many, relevant for WH_MANAGER scoping

8 Order Lifecycle & State Machine

8.1 Order-Level States



8.2 Allocation-Level States (per OrderAllocation row)

ALLOCATED → PICKED → SHIPPED

An order only advances to SHIPPED once **every** OrderAllocation under it is SHIPPED — this correctly models a split order where two warehouses ship at different times.

8.3 Transition Table

From	To	Trigger	Guard
CREATED	ALLOCATED	routing engine fully covers all lines	all lines fully allocated
CREATED	PARTIALLY_ALLOCATED	routing engine partially covers lines	≥1 line has uncovered quantity
ALLOCATED / PARTIALLY_ALLOCATED	CONFIRMED	WH_MANAGER confirms	role check
CONFIRMED	PICKED	all allocations marked picked	role check, allocation states
PICKED	SHIPPED	all allocations marked shipped	role check, allocation states
SHIPPED	DELIVERED	delivery confirmation (manual or webhook)	—
CREATED / ALLOCATED / PARTIALLY_ALLOCATED / CONFIRMED / PICKED	CANCELLED	ADMIN cancels	releases reservedQuantity back to StockItem

8.4 Why a State Pattern, Not an if/else Chain

A naive implementation checks `if (order.getStatus() == SHIPPED) { ... } else if (...)` scattered across the service layer. This has two problems: the transition rules live in

multiple places, and nothing stops an invalid transition from being written by mistake in some new code path six months later.

Instead, each status maps to a `OrderState` implementation with a single method: `OrderState transitionTo(OrderEvent event)`, which either returns the next valid state or throws `IllegalStateException`. This makes illegal transitions a *compile-time-adjacent* concept (a missing case in the state's transition map) rather than a runtime surprise, and it is the textbook definition of the **State design pattern** — worth naming explicitly in the report.

```
public interface OrderState {
    OrderState onEvent(OrderEvent event);
    OrderStatus status();
}

public final class CreatedState implements OrderState {
    @Override
    public OrderState onEvent(OrderEvent event) {
        return switch (event) {
            case ROUTED_FULL -> new AllocatedState();
            case ROUTED_PARTIAL -> new PartiallyAllocatedState();
            case CANCEL -> new CancelledState();
            default -> throw new IllegalStateException(status(), event);
        };
    }
    @Override public OrderStatus status() { return OrderStatus.CREATED; }
}
```

9 REST API Design

All endpoints are prefixed `/api/v1`. Authentication via Authorization: Bearer <jwt> except `/auth/**`.

9.1 Auth

Method	Endpoint	Role	Description
POST	<code>/auth/register</code>	public	create a user (role assigned by an existing ADMIN in practice, or seeded)
POST	<code>/auth/login</code>	public	returns access + refresh token
POST	<code>/auth/refresh</code>	public (valid refresh token)	rotates access token

9.2 Warehouses

Method	Endpoint	Role	Description
GET	<code>/warehouses</code>	any authenticated	list, paginated
POST	<code>/warehouses</code>	ADMIN	create
PUT	<code>/warehouses/{id}</code>	ADMIN	update
DELETE	<code>/warehouses/{id}</code>	ADMIN	soft-deactivate

9.3 Products

Method	Endpoint	Role	Description
GET	<code>/products</code>	any authenticated	list, paginated, filter by category
POST	<code>/products</code>	ADMIN, SUPPLIER	create
PUT	<code>/products/{id}</code>	ADMIN, SUPPLIER	update

9.4 Stock

Method	Endpoint	Role	Description
GET	<code>/stock</code>	any authenticated	filter by warehouseId, productId, belowThreshold

Method	Endpoint	Role	Description
POST	/stock	WH_MANAGER	create/adjust a StockItem
PATCH	/stock/{id}/quantity	WH_MANAGER	manual quantity correction (audited)

9.5 Orders (the core flow)

Method	Endpoint	Role	Description
POST	/orders	any authenticated	places an order; triggers routing synchronously; returns the allocation plan
GET	/orders	any authenticated (scoped)	paginated, filter by status/date/customer
GET	/orders/{id}	any authenticated (scoped)	full detail including allocations and score breakdown
POST	/orders/{id}/confirm	WH_MANAGER	CREATED/ALLOCATED to CONFIRMED
POST	/orders/{id}/allocations/{allocationId}/pick	WH_MANAGER	marks one allocation PICKED
POST	/orders/{id}/allocations/{allocationId}/ship	WH_MANAGER	marks one allocation SHIPPED
POST	/orders/{id}/cancel	ADMIN	cancels, releases reserved stock

9.5.1 Example — POST /orders request

```
{
  "customerId": "cle6a9f0-...",
  "lines": [
    { "productId": "p-9931", "quantity": 40 },
    { "productId": "p-1042", "quantity": 15 }
  ]
}
```

9.5.2 Example — POST /orders response

```

{
  "orderId": "o-7712",
  "status": "PARTIALLY_ALLOCATED",
  "lines": [
    {
      "productId": "p-9931",
      "quantityRequested": 40,
      "allocations": [
        {
          "warehouseId": "w-BLR-01",
          "quantityAllocated": 30,
          "scoreBreakdown": { "normalizedDistance": 0.12, "normalizedCost": 0.30, "shortfallPenalty": 0.0
        },
        {
          "warehouseId": "w-BLR-04",
          "quantityAllocated": 10,
          "scoreBreakdown": { "normalizedDistance": 0.41, "normalizedCost": 0.20, "shortfallPenalty": 0.2
        }
      ],
      "backorderedQuantity": 0
    },
    {
      "productId": "p-1042",
      "quantityRequested": 15,
      "allocations": [],
      "backorderedQuantity": 15
    }
  ]
}

```

9.6 Error Response Shape (consistent across all endpoints)

```

{
  "timestamp": "2026-08-12T10:15:30Z",
  "status": 409,
  "error": "STOCK_CONFLICT",
  "message": "Unable to allocate stock after 3 retries due to high contention",
  "path": "/api/v1/orders"
}

```

10 Core Algorithm Deep Dive: Fulfillment Routing

This is the section that differentiates STRIDE from a tutorial project. Read it alongside the code in Section 10.7 — the two are meant to be cross-referenced.

10.1 Problem Formulation

Given an order with lines $L = \{l_1, l_2, \dots, l_n\}$, each requiring quantity q_i of product p_i , and a set of warehouses W each holding some quantity of some products, find an assignment of (warehouse, line, quantity) triples such that:

- For each line, the sum of allocated quantities does not exceed the requested quantity, and does not exceed available stock at any warehouse.
- The total “cost” of the assignment — a weighted function of distance, warehouse cost factor, shortfall, and shipment fragmentation — is minimized.
- Where full coverage is impossible, the plan explicitly reports the uncovered (backordered) quantity per line.

This is a variant of the **multi-item, multi-source transportation problem**, which in its general form is solvable optimally via **linear programming** (or, for small integer cases, via **Integer Linear Programming**, which is NP-hard in general). STRIDE deliberately does not implement a full LP/ILP solver — Section 10.6 justifies why.

10.2 Why Not Just Solve the ILP?

An exact ILP formulation would look like: minimize $\sum_{i,j} c_{ij}x_{ij}$ subject to supply and demand constraints, where x_{ij} is the quantity of line i served by warehouse j . This is exactly correct — but:

1. **Runtime is not bounded.** ILP solve time can spike unpredictably depending on problem structure. An order-placement API is a synchronous, user-facing call; an unbounded solve time is not acceptable (violates NFR-1’s implicit real-time requirement).
2. **It is disproportionate to the problem size.** Real orders have a handful of lines and a bounded number of realistically-relevant nearby warehouses (most warehouses are irrelevant to a given customer simply by geography). The overhead of embedding and maintaining an LP solver (e.g., OR-Tools) is not justified by the problem size actually encountered.
3. **It sacrifices explainability.** NFR-2 requires that each allocation decision can be explained via its score breakdown. An LP solver’s output is a global optimum with no per-decision “why,” which is worse for both the demo and for a warehouse manager who needs to trust the system.

Decision: implement a **two-phase heuristic** — greedy per-line scoring, followed by a bounded bitmask DP consolidation pass — that is fast, explainable, and provably good enough for the realistic candidate-set sizes involved. This tradeoff, stated explicitly, is the single most important paragraph to have memorized for a viva.

10.3 Phase 1 — Candidate Filtering

Before scoring anything, filter warehouses down to a realistic candidate set, for two reasons: correctness (a warehouse with zero stock of the product is irrelevant) and performance (bounding k before the DP step in Phase 3 is what keeps 2^k tractable).

```
candidates(line) = warehouses W such that:  
    W.active == true  
    AND StockItem(W, line.product).availableToPromise > 0  
    AND distance(W, customer) <= MAX_CANDIDATE_RADIUS_KM    // configurable, default 300km  
ORDER BY distance ASC  
LIMIT MAX_CANDIDATES_PER_LINE    // default 15
```

10.4 Phase 2 — Per-Line Greedy Scoring

For each order line, each remaining candidate warehouse is scored:

$$\text{score}(w) = w_1 \cdot \widehat{\text{dist}}(w) + w_2 \cdot \widehat{\text{cost}}(w) + w_3 \cdot \text{shortfall}(w)$$

where $\widehat{\text{dist}}$ and $\widehat{\text{cost}}$ are **min-max normalized within the candidate batch** (not globally — a warehouse that is “far” relative to a nearby cluster of candidates should score worse than one that is “far” in absolute terms but still the closest option available):

$$\widehat{\text{dist}}(w) = \begin{cases} 0.5 & \text{if } \max(\text{dist}) = \min(\text{dist}) \text{ across candidates} \\ \frac{\text{dist}(w) - \min(\text{dist})}{\max(\text{dist}) - \min(\text{dist})} & \text{otherwise} \end{cases}$$

The 0.5 fallback avoids a division-by-zero and deliberately encodes “no signal, stay neutral” rather than defaulting to 0 (which would wrongly favor every warehouse equally as “best”) or 1 (which would wrongly penalize every warehouse as “worst”). The identical formula applies to $\widehat{\text{cost}}$.

$$\text{shortfall}(w) = \max \left(0, \frac{\text{needed} - \text{available}(w)}{\text{needed}} \right)$$

Shortfall is 0 when a warehouse can fully cover the line alone, and approaches 1 as its available stock covers less and less of what’s needed — this is what discourages the greedy pass from spraying an order across many small partial allocations when one or two warehouses would do.

Default weights: $w_1 = 0.4$, $w_2 = 0.3$, $w_3 = 0.3$ — configurable via `application.yml`, and explicitly called out in the report as a tunable business parameter, not a hardcoded constant.

Greedy allocation: sort candidates ascending by score; allocate from the best-scoring warehouse first, consuming its available stock, moving to the next warehouse

only if the line’s requested quantity is not yet fully covered. Stop when covered or candidates are exhausted (remaining quantity becomes `backorderedQuantity`).

10.5 Phase 3 — Basket-Level Consolidation (Bitmask DP)

Line-by-line greedy can fragment an order unnecessarily: line 1 goes to Warehouse A, line 2 goes to Warehouse B, when Warehouse C could have handled both lines at a slightly higher individual score but with only **one** shipment instead of two. This phase corrects for that.

Let the union of candidate warehouses across *all* lines in the order be $C = \{c_1, \dots, c_k\}$, with k bounded to at most 15 by the Phase 1 filter (so $2^k \leq 32768$ — trivially fast). For each subset $S \subseteq C$ (represented as a bitmask), determine whether S can cover the entire order (every line’s requested quantity is satisfiable using only warehouses in S), and if so compute:

$$\text{totalScore}(S) = \sum_{w \in S} \sum_{\text{lines served by } w} \text{score}(w, \text{line}) - \beta \cdot f(|S|)$$

where $f(|S|)$ is a **consolidation bonus** that grows as the number of distinct warehouses in the subset shrinks (e.g., $f(|S|) = (k_{\max} - |S|)$), and β is a tunable weight controlling how strongly the system prefers fewer shipments over marginally better individual line scores.

```
for mask in 1 .. (1 << k) - 1:
    subset = warehousesInMask(mask, candidates)
    if canCoverAllLines(subset, orderLines):          # feasibility check, O(L) per mask
        plan = bestAllocationWithinSubset(subset, orderLines)
        score = totalScore(plan) - consolidationBonus(popcount(mask))
        if score < bestScoreSoFar:
            bestScoreSoFar = score
            bestPlan = plan
```

Complexity: $O(2^k \cdot L \cdot \bar{c})$ where L is the number of order lines and $\bar{c}$ is the average candidates per line within the subset — bounded and small in practice given $k \leq 15$ and L typically in the single digits for a retail order. This is stated explicitly as **exponential in the candidate set size**, with the mitigation being the Phase 1 pre-filter that bounds k — a sentence worth having ready verbatim for a viva question about scalability.

10.6 Complexity Summary

Phase	Technique	Complexity
1. Candidate filtering	sort + limit	$O(n \log n)$ per line, n = warehouses stocking that product

Phase	Technique	Complexity
2. Per-line greedy	min-max normalize + sort + greedy consume	$O(k \log k)$ per line
3. Basket consolidation	bitmask DP over candidate subsets	$O(2^k \cdot L)$, $k \leq 15$ (bounded by Phase 1)
Alternative (rejected)	exact ILP	worst case exponential, no runtime guarantee

10.7 Reference Implementation (Java)

```

public record StockSnapshot(UUID warehouseId, UUID productId, int available,
                           double latitude, double longitude, double costFactor) {}

public record LineRequest(UUID productId, int quantityRequested) {}

public record ScoreBreakdown(double normalizedDistance, double normalizedCost, double shortfall) {
    double total(double w1, double w2, double w3) {
        return w1 * normalizedDistance + w2 * normalizedCost + w3 * shortfall;
    }
}

public record AllocationEntry(UUID warehouseId, UUID productId, int quantity, ScoreBreakdown breakdown) {}

public record AllocationPlan(List<AllocationEntry> entries, Map<UUID, Integer> backorderedByProduct) {}

public interface RoutingStrategy {
    AllocationPlan route(Customer customer, List<LineRequest> lines, List<StockSnapshot> candidates);
}

public final class DistanceCostStrategy implements RoutingStrategy {

    private static final double W_DIST = 0.4, W_COST = 0.3, W_SHORTFALL = 0.3;
    private static final int MAX_CANDIDATES_PER_LINE = 15;

    @Override
    public AllocationPlan route(Customer customer, List<LineRequest> lines, List<StockSnapshot> allCandidates) {
        // Phase 1 + 2: per-line greedy (fallback / baseline plan)
        List<AllocationEntry> greedyEntries = new ArrayList<>();
        Map<UUID, Integer> backordered = new HashMap<>();

        for (LineRequest line : lines) {
            List<StockSnapshot> candidates = allCandidates.stream()
                .filter(s -> s.productId().equals(line.productId()) && s.available() > 0)
                .sorted(Comparator.comparingDouble(s -> distance(s, customer)))
                .limit(MAX_CANDIDATES_PER_LINE)
                .toList();

```

```

    int remaining = line.quantityRequested();
    double minDist = candidates.stream().mapToDouble(s -> distance(s, customer)).min().orElse(0);
    double maxDist = candidates.stream().mapToDouble(s -> distance(s, customer)).max().orElse(0);
    double minCost = candidates.stream().mapToDouble(StockSnapshot::costFactor).min().orElse(0);
    double maxCost = candidates.stream().mapToDouble(StockSnapshot::costFactor).max().orElse(0);

    List<StockSnapshot> ranked = candidates.stream()
        .sorted(Comparator.comparingDouble(s ->
            scoreOf(s, customer, remaining, minDist, maxDist, minCost, maxCost)))
        .toList();

    for (StockSnapshot s : ranked) {
        if (remaining <= 0) break;
        int take = Math.min(remaining, s.available());
        if (take <= 0) continue;
        ScoreBreakdown breakdown = breakdownOf(s, customer, remaining, minDist, maxDist, minCost,
            greedyEntries.add(new AllocationEntry(s.warehouseId(), s.productId(), take, breakdown));
        remaining -= take;
    }
    if (remaining > 0) {
        backordered.merge(line.productId(), remaining, Integer::sum);
    }
}

AllocationPlan greedyPlan = new AllocationPlan(greedyEntries, backordered);

// Phase 3: attempt consolidation, keep it only if it doesn't increase backorder
// and produces a lower total score (see ConsolidationPlanner, Section 10.8)
AllocationPlan consolidated = ConsolidationPlanner.tryConsolidate(customer, lines, allCandidates,
    return consolidated != null ? consolidated : greedyPlan;
}

private double distance(StockSnapshot s, Customer c) {
    return haversineKm(s.latitude(), s.longitude(), c.getLatitude(), c.getLongitude());
}

private double scoreOf(StockSnapshot s, Customer c, int needed,
    double minDist, double maxDist, double minCost, double maxCost) {
    double dNorm = (maxDist == minDist) ? 0.5 : (distance(s, c) - minDist) / (maxDist - minDist);
    double cNorm = (maxCost == minCost) ? 0.5 : (s.costFactor() - minCost) / (maxCost - minCost);
    double shortfall = Math.max(0, (needed - s.available()) / (double) needed);
    return W_DIST * dNorm + W_COST * cNorm + W_SHORTFALL * shortfall;
}

private ScoreBreakdown breakdownOf(StockSnapshot s, Customer c, int needed,
    double minDist, double maxDist, double minCost, double maxCost) {
    double dNorm = (maxDist == minDist) ? 0.5 : (distance(s, c) - minDist) / (maxDist - minDist);

```

```

        double cNorm = (maxCost == minCost) ? 0.5 : (s.costFactor() - minCost) / (maxCost - minCost);
        double shortfall = Math.max(0, (needed - s.available()) / (double) needed);
        return new ScoreBreakdown(dNorm, cNorm, shortfall);
    }

    static double haversineKm(double lat1, double lon1, double lat2, double lon2) {
        final double R = 6371.0;
        double dLat = Math.toRadians(lat2 - lat1);
        double dLon = Math.toRadians(lon2 - lon1);
        double a = Math.sin(dLat / 2) * Math.sin(dLat / 2)
            + Math.cos(Math.toRadians(lat1)) * Math.cos(Math.toRadians(lat2))
            * Math.sin(dLon / 2) * Math.sin(dLon / 2);
        return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    }
}

```

10.8 Consolidation Planner (Bitmask DP)

```

public final class ConsolidationPlanner {

    private static final double CONSOLIDATION_BETA = 0.05; // tunable

    public static AllocationPlan tryConsolidate(Customer customer, List<LineRequest> lines,
                                                List<StockSnapshot> allCandidates, AllocationPlan baseline) {
        List<UUID> candidateWarehouses = allCandidates.stream()
            .map(StockSnapshot::warehouseId).distinct().limit(15).toList();
        int k = candidateWarehouses.size();
        if (k == 0 || k > 15) return null; // guard: skip DP if pre-filter failed to bound k

        double bestScore = Double.MAX_VALUE;
        AllocationPlan bestPlan = null;

        for (int mask = 1; mask < (1 << k); mask++) {
            List<UUID> subset = warehousesInMask(mask, candidateWarehouses);
            AllocationPlan attempt = allocateWithinSubset(customer, lines, allCandidates, subset);
            if (attempt == null) continue; // subset cannot cover the order at all – skip
            if (totalBackordered(attempt) > totalBackordered(baseline)) continue; // never worsen coverage

            double score = totalRawScore(attempt) - CONSOLIDATION_BETA * (15 - Integer.bitCount(mask));
            if (score < bestScore) {
                bestScore = score;
                bestPlan = attempt;
            }
        }
        return bestPlan;
    }
}

```

```

private static List<UUID> warehousesInMask(int mask, List<UUID> all) {
    List<UUID> result = new ArrayList<>();
    for (int i = 0; i < all.size(); i++) {
        if ((mask & (1 << i)) != 0) result.add(all.get(i));
    }
    return result;
}

// allocateWithinSubset: same greedy scoring as Phase 2, but candidates filtered to `subset` only.
// totalBackordered / totalRawScore: simple aggregation helpers over AllocationPlan.
}

```

11 Object-Oriented Design & Design Patterns

Every pattern below is tied to a concrete problem it solves in this codebase — none are included decoratively.

11.1 Strategy Pattern — `RoutingStrategy`

Problem it solves: the scoring formula (Section 10.4) is a business decision that will change — different weight tunings, or eventually a completely different algorithm (e.g., an ILP-backed strategy for offline batch re-optimization). Hardcoding the formula inside `OrderService` would mean every future change touches orchestration code.

Solution: `FulfillmentRoutingService` depends on the `RoutingStrategy` interface (Section 10.7), injected via Spring. Swapping `DistanceCostStrategy` for a different implementation requires zero changes outside a configuration bean.

11.2 State Pattern — `OrderState`

Covered in Section 8.4. Solves: preventing illegal order-status transitions by encoding the transition table into the type system rather than scattering `if/else` checks across the service layer.

11.3 Builder Pattern — `AllocationPlan` construction

Problem it solves: an `AllocationPlan` is built incrementally across Phase 1–3 of the routing algorithm, from multiple partial results (per-line entries, backorder map, score breakdowns). Constructing it via a mutable builder keeps the intermediate state contained and the final object immutable once returned.

```
public final class AllocationPlanBuilder {
    private final List<AllocationEntry> entries = new ArrayList<>();
    private final Map<UUID, Integer> backordered = new HashMap<>();

    public AllocationPlanBuilder addEntry(AllocationEntry entry) { entries.add(entry); return this; }
    public AllocationPlanBuilder addBackorder(UUID productId, int qty) {
        backordered.merge(productId, qty, Integer::sum); return this;
    }
    public AllocationPlan build() {
        return new AllocationPlan(List.copyOf(entries), Map.copyOf(backordered));
    }
}
```

11.4 Single Responsibility Principle — the three-way split

`FulfillmentRoutingService` (decides), `StockAllocationExecutor` (persists + retries), `OrderService` (orchestrates + enforces state transitions) each have exactly one reason to change:

- Routing logic changes → only FulfillmentRoutingService/RoutingStrategy change.
- Persistence/locking strategy changes → only StockAllocationExecutor changes.
- Business workflow changes (e.g., a new approval step) → only OrderService changes.

This is the detail evaluators specifically look for when they ask “is this all crammed into one controller?” — being able to point at three distinctly-responsible classes, each independently testable, is the concrete answer.

11.5 Composition Over Inheritance

Warehouse **has-a** location (latitude/longitude) and **has-a** collection of StockItem; there is no Location or Inventory base class being inherited from. The domain does not have a natural “is-a” hierarchy among Warehouse/Product/StockItem, so forcing one in would only add fragility (the classic diamond-inheritance or fragile-base-class problems) without benefit. This is worth stating explicitly, since students are sometimes evaluated on *whether they knew when not to use inheritance*, not just whether they used it.

11.6 Repository Pattern (via Spring Data JPA)

Each aggregate root (Order, Warehouse, Product, StockItem, User) has its own JpaRepository, keeping persistence access uniform and query logic declarative (findByWarehouseIdAndProductId, etc.) rather than hand-rolled JDBC scattered through the service layer.

12 Concurrency & Data Consistency

12.1 The Race Condition

Two customers order the last unit of a product from the same warehouse at nearly the same moment. Both requests read `StockItem.quantity = 1`. Both compute a valid allocation of 1 unit. Both attempt to decrement to 0. Without protection, **both succeed**, and the warehouse has oversold by one unit — a real, common bug in systems that “work” in a demo but fail under load.

12.2 Why Optimistic Locking, Not Pessimistic

Approach	Behavior	Tradeoff
Pessimistic (SELECT ... FOR UPDATE)	First transaction locks the row; others block until it commits/rolls back	Simple to reason about, but serializes <i>all</i> writers to a given <code>StockItem</code> , even when most requests wouldn't actually conflict — poor throughput under load
Optimistic (@Version)	Every transaction reads freely; on write, the DB checks the version column hasn't changed since read; if it has, the write fails and the caller retries	Higher throughput in the common case (no conflict); pays a retry cost only when contention actually happens — matches the expected access pattern (most orders don't compete for the exact same stock row)

```
@Entity
public class StockItem {
    @Id @GeneratedValue private UUID id;
    // ... other fields
    @Version
    private long version;
}
```

Under the hood, JPA/Hibernate issues:

```
UPDATE stock_item SET quantity = ?, reserved_quantity = ?, version = version + 1
WHERE id = ? AND version = ?;
```

If the WHERE clause matches zero rows (because another transaction already bumped the version), Hibernate throws `ObjectOptimisticLockingFailureException`.

12.3 Retry Scope — the Detail Most Implementations Get Wrong

A common mistake: wrap only the repository `save()` call in a retry. This fails, because the **in-memory entity being saved still reflects the stale read** — retrying the save just re-attempts writing the same stale data and fails identically every time (or worse, silently succeeds with wrong values if the version check is bypassed).

Correct scope: the retry must re-run the *entire* use case — re-fetch fresh stock, re-run the (pure, side-effect-free) routing algorithm against that fresh snapshot, and only then attempt to persist:

```
@Service
public class OrderService {

    @Retryable(
        retryFor = ObjectOptimisticLockingFailureException.class,
        maxAttempts = 3,
        backoff = @Backoff(delay = 50, multiplier = 2)
    )
    @Transactional
    public OrderResponse placeOrder(OrderRequest request) {
        List<StockSnapshot> freshStock = stockRepository.findCandidateSnapshots(request.lines());
        AllocationPlan plan = routingService.route(request.customer(), request.lines(), freshStock);
        Order order = allocationExecutor.persistOrderWithAllocations(request, plan); // throws on version
        return OrderMapper.toResponse(order);
    }

    @Recover
    public OrderResponse recoverFromContention(ObjectOptimisticLockingFailureException ex, OrderRequest request) {
        throw new StockConflictException("Unable to allocate stock after repeated contention", ex);
    }
}
```

The `@Recover` method matters as much as the `@Retryable` — after exhausting retries, the system must fail loudly and clearly (409 STOCK_CONFLICT) rather than silently corrupting data or hanging.

12.4 Reserved vs. Available Quantity

As introduced in Section 7.2.3, `StockItem` tracks quantity (physical) and `reservedQuantity` (promised-but-not-shipped) separately. This means:

- **On allocation:** `reservedQuantity += allocatedAmount` (physical quantity untouched — nothing has left the building yet).
- **On shipment:** `quantity -= allocatedAmount`; `reservedQuantity -= allocatedAmount` (now it actually leaves).
- **On cancellation before shipment:** `reservedQuantity -= allocatedAmount` (release the hold).

Available-to-promise for routing purposes is always `quantity - reservedQuantity`, not raw

quantity — this is what prevents a second order from allocating stock that's already promised to a first, unshipped order.

12.5 Isolation Level

`READ COMMITTED` (Postgres default) is sufficient given the explicit optimistic-locking design — the `@Version` check substitutes for stronger isolation guarantees at the specific row level that matters, without paying for full `SERIALIZABLE` isolation overhead across unrelated transactions.

13 Security Design

13.1 Authentication Flow

1. POST /auth/login — credentials validated against passwordHash (BCrypt, strength 12).
2. On success, issue a short-lived **access token** (JWT, 15 min expiry) and a longer-lived **refresh token** (7 days, stored hashed server-side to allow revocation).
3. Every subsequent request includes Authorization: Bearer <access_token>.
4. A stateless JwtFilter (extends OncePerRequestFilter) validates the signature and expiry, then populates the SecurityContext.

```
public class JwtFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            String token = header.substring(7);
            if (jwtService.isValid(token)) {
                var auth = jwtService.buildAuthentication(token);
                SecurityContextHolder.getContext().setAuthentication(auth);
            }
        }
        chain.doFilter(req, res);
    }
}
```

13.2 Authorization — Method-Level, Not Just URL-Level

```
@PreAuthorize("hasRole('WH_MANAGER') and @warehouseGuard.canAccess(#warehouseId, principal)")
@PatchMapping("/stock/{id}/quantity")
public StockItemResponse adjustQuantity(@PathVariable UUID id, @RequestBody AdjustQuantityRequest req) {
```

@warehouseGuard.canAccess(...) is a custom Spring bean checked inside the SpEL expression — it enforces FR-1.3 (a manager may only touch warehouses they're assigned to), which a blanket hasRole() check alone cannot express.

13.3 Password & Secret Handling

- Passwords hashed with BCrypt before storage; plaintext never logged.
- JWT signing secret loaded from environment variable, never committed — .env is git-ignored, .env.example documents required keys.
- Refresh tokens stored as a hash (not plaintext) so a database leak doesn't directly yield usable tokens.

14 Database Schema (Flyway Migration)

```
-- V1__init_schema.sql

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email VARCHAR(255) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role VARCHAR(30) NOT NULL CHECK (role IN ('ADMIN', 'WH_MANAGER', 'SUPPLIER')),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE warehouse (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(255) UNIQUE NOT NULL,
  address VARCHAR(500),
  latitude DOUBLE PRECISION NOT NULL,
  longitude DOUBLE PRECISION NOT NULL,
  cost_factor DOUBLE PRECISION NOT NULL DEFAULT 1.0,
  active BOOLEAN NOT NULL DEFAULT true,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE warehouse_manager (
  warehouse_id UUID NOT NULL REFERENCES warehouse(id),
  user_id UUID NOT NULL REFERENCES users(id),
  PRIMARY KEY (warehouse_id, user_id)
);

CREATE TABLE product (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sku VARCHAR(100) UNIQUE NOT NULL,
  name VARCHAR(255) NOT NULL,
  category VARCHAR(100),
  unit_weight_kg DOUBLE PRECISION,
  reorder_threshold INT NOT NULL DEFAULT 10
);

CREATE TABLE stock_item (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  warehouse_id UUID NOT NULL REFERENCES warehouse(id),
  product_id UUID NOT NULL REFERENCES product(id),
  quantity INT NOT NULL CHECK (quantity >= 0),
  reserved_quantity INT NOT NULL DEFAULT 0 CHECK (reserved_quantity >= 0),
  batch_expiry DATE,
  version BIGINT NOT NULL DEFAULT 0,
```

```

    UNIQUE (warehouse_id, product_id)
);

CREATE TABLE customer (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255),
    phone VARCHAR(30),
    address VARCHAR(500),
    latitude DOUBLE PRECISION NOT NULL,
    longitude DOUBLE PRECISION NOT NULL
);

CREATE TABLE orders (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL REFERENCES customer(id),
    status VARCHAR(30) NOT NULL DEFAULT 'CREATED',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE order_line (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_id UUID NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES product(id),
    quantity_requested INT NOT NULL CHECK (quantity_requested > 0)
);

CREATE TABLE order_allocation (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    order_line_id UUID NOT NULL REFERENCES order_line(id) ON DELETE CASCADE,
    warehouse_id UUID NOT NULL REFERENCES warehouse(id),
    quantity_allocated INT NOT NULL CHECK (quantity_allocated > 0),
    score_breakdown JSONB,
    status VARCHAR(20) NOT NULL DEFAULT 'ALLOCATED'
);

CREATE INDEX idx_stock_item_warehouse_product ON stock_item(warehouse_id, product_id);
CREATE INDEX idx_order_status ON orders(status);
CREATE INDEX idx_order_line_order ON order_line(order_id);
CREATE INDEX idx_allocation_order_line ON order_allocation(order_line_id);
CREATE INDEX idx_stock_reorder_check ON stock_item(product_id) WHERE reserved_quantity > 0;

```

Design notes: - UNIQUE (warehouse_id, product_id) on stock_item guarantees one row per warehouse/product pair — the natural key the whole routing algorithm assumes. - CHECK (quantity >= 0) and CHECK (reserved_quantity >= 0) are a second line of defense against bugs in the application layer — the database itself refuses to go negative,

even if a future code change accidentally bypasses the optimistic-locking path. -
score_breakdown JSONB directly supports NFR-2 (explainability) — the *why* behind every
allocation decision is queryable, not just logged and discarded.

15 DSA Concepts Applied — Summary Table

Concept	Where	Complexity	Why this technique specifically
Greedy algorithm	Per-line warehouse selection (Section 10.4)	$O(k \log k)$ per line	Locally optimal choice is cheap to compute and provably reasonable when the shortfall penalty discourages fragmentation
Bitmask dynamic programming	Basket-level consolidation (Section 10.5)	$O(2^k \cdot L)$, $k \leq 15$	Exact search over a <i>bounded</i> subset space — correct and fast because k is capped by the candidate pre-filter
Min-max normalization	Score computation (Section 10.4)	$O(k)$ per batch	Keeps distance and cost comparable on the same 0-1 scale regardless of their differing units/ranges
Hashing (HashMap)	(warehouseId, productId) → StockItem lookup during a routing pass	$O(1)$ average lookup	Avoids repeated linear scans over stock while scoring multiple lines against the same candidate set
Priority queue (optional refinement)	Top-k candidate selection when candidate lists are large	$O(n \log k)$ vs. $O(n \log n)$ full sort	Only worth it if MAX_CANDIDATES_PER_LINE is small relative to total warehouses — documented as a possible refinement in Future Work
State machine / finite automaton	Order lifecycle (Section 8)	$O(1)$ transition lookup	Encodes legality of transitions structurally; prevents an entire category of bugs by construction

Concept	Where	Complexity	Why this technique specifically
Exponential backoff	Optimistic-lock retry (Section 12.3)	bounded by <code>maxAttempts</code>	Standard technique for retrying under contention without hammering the database immediately
Haversine distance	Geographic scoring	$O(1)$ per pair	Standard great-circle distance formula, accurate enough at warehouse-to-customer scale (city/regional distances)

16 Testing Strategy

16.1 Philosophy

Tests are prioritized by risk, not by ease of writing. The routing algorithm and the concurrency behavior are where real bugs hide and where a tutorial-level project would have zero coverage — so those get the deepest tests. CRUD endpoints get lighter, representative coverage.

16.2 Unit Tests — Pure Algorithm (no Spring context)

```
class DistanceCostStrategyTest {

    private final DistanceCostStrategy strategy = new DistanceCostStrategy();

    @Test
    void fullyAllocatesFromSingleWarehouseWhenSufficient() {
        var customer = testCustomer(12.97, 77.59); // Bengaluru
        var lines = List.of(new LineRequest(PRODUCT_A, 20));
        var stock = List.of(
            snapshot(WAREHOUSE_1, PRODUCT_A, 50, 12.97, 77.59, 1.0),
            snapshot(WAREHOUSE_2, PRODUCT_A, 50, 13.30, 77.10, 1.2) // farther, costlier
        );

        AllocationPlan plan = strategy.route(customer, lines, stock);

        assertThat(plan.entries().hasSize(1);
        assertThat(plan.entries().get(0).warehouseId()).isEqualTo(WAREHOUSE_1);
        assertThat(plan.entries().get(0).quantity()).isEqualTo(20);
        assertThat(plan.backorderedByProduct()).isEmpty();
    }

    @Test
    void splitsAcrossWarehousesWhenNoSingleWarehouseSuffices() {
        var lines = List.of(new LineRequest(PRODUCT_A, 40));
        var stock = List.of(
            snapshot(WAREHOUSE_1, PRODUCT_A, 25, 12.97, 77.59, 1.0),
            snapshot(WAREHOUSE_2, PRODUCT_A, 30, 12.98, 77.60, 1.0)
        );

        AllocationPlan plan = strategy.route(testCustomer(12.97, 77.59), lines, stock);

        int totalAllocated = plan.entries().stream().mapToInt(AllocationEntry::quantity).sum();
        assertThat(totalAllocated).isEqualTo(40);
        assertThat(plan.entries().hasSizeGreaterThanOrEqualTo(2);
    }
}
```



```

@Test
void reportsBackorderWhenTotalStockInsufficient() {
    var lines = List.of(new LineRequest(PRODUCT_A, 100));
    var stock = List.of(snapshot(WAREHOUSE_1, PRODUCT_A, 30, 12.97, 77.59, 1.0));

    AllocationPlan plan = strategy.route(testCustomer(12.97, 77.59), lines, stock);

    assertThat(plan.backorderedByProduct()).containsEntry(PRODUCT_A, 70);
}

@Test
void handlesEquidistantWarehousesWithoutDivisionByZero() {
    var lines = List.of(new LineRequest(PRODUCT_A, 10));
    var stock = List.of(
        snapshot(WAREHOUSE_1, PRODUCT_A, 20, 12.97, 77.59, 1.0),
        snapshot(WAREHOUSE_2, PRODUCT_A, 20, 12.97, 77.59, 1.0) // identical location
    );

    assertThatCode(() -> strategy.route(testCustomer(12.97, 77.59), lines, stock))
        .doesNotThrowAnyException();
}
}

```

ConsolidationPlannerTest mirrors this structure, with a specific case asserting that a 2-line order which *could* be split for a marginally better per-line score instead consolidates onto a single warehouse when CONSOLIDATION_BETA makes that worthwhile — this is the test that directly proves Section 10.5’s design intent, not just “code runs without exceptions.”

16.3 Integration Tests — Testcontainers

```

@SpringBootTest
@Testcontainers
class OrderPlacementIntegrationTest {

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:15")
        .withDatabaseName("smartroute_test");

    @DynamicPropertySource
    static void configure(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
    }

    @Autowired MockMvc mockMvc;
}

```

```

@Autowired StockItemRepository stockItemRepository;

@Test
void placingOrderReservesStockAndReturnsAllocationPlan() throws Exception {
    seedWarehouseWithStock(WAREHOUSE_1, PRODUCT_A, 50);

    mockMvc.perform(post("/api/v1/orders")
        .contentType(APPLICATION_JSON)
        .content(orderRequestJson(PRODUCT_A, 20)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.status").value("ALLOCATED"))
        .andExpect(jsonPath("$.lines[0].allocations[0].quantityAllocated").value(20));

    StockItem stock = stockItemRepository.findByWarehouseAndProduct(WAREHOUSE_1, PRODUCT_A);
    assertThat(stock.getReservedQuantity()).isEqualTo(20);
}
}

```

16.4 Concurrency Test — the Most Important Test in the Project

This test directly proves NFR-1. It deliberately fires concurrent requests at the same scarce stock row and asserts that the total allocated never exceeds what was available — this is the test to demo live in a viva if time allows.

```

@Test
void concurrentOrdersForLastUnitsNeverOversell() throws Exception {
    seedWarehouseWithStock(WAREHOUSE_1, PRODUCT_A, 10); // exactly 10 units available

    int numberOfOrders = 20; // more requests than stock – guarantees contention
    ExecutorService pool = Executors.newFixedThreadPool(10);
    CountDownLatch ready = new CountDownLatch(numberOfOrders);
    CountDownLatch go = new CountDownLatch(1);

    List<Future<Integer>> results = new ArrayList<>();
    for (int i = 0; i < numberOfOrders; i++) {
        results.add(pool.submit(() -> {
            ready.countDown();
            go.await();
            try {
                var response = orderService.placeOrder(singleLineOrder(PRODUCT_A, 1));
                return totalAllocated(response);
            } catch (StockConflictException e) {
                return 0; // acceptable – rejected under contention, did NOT oversell
            }
        }));
    }

    ready.await();
    go.countDown(); // release all threads simultaneously
}

```

```

int totalAllocated = 0;
for (Future<Integer> f : results) totalAllocated += f.get();

assertThat(totalAllocated).isLessThanOrEqualTo(10); // the property that actually matters

StockItem finalStock = stockItemRepository.findByWarehouseAndProduct(WAREHOUSE_1, PRODUCT_A);
assertThat(finalStock.getReservedQuantity()).isEqualTo(totalAllocated); // consistency check
}

```

16.5 Coverage Targets

Area	Target	Rationale
Routing / consolidation algorithm	100% branch coverage	Highest-risk, highest-value code in the system
Order state machine	100% of transition table	Illegal transitions must be provably impossible
Controllers	Happy path + one failure case per endpoint	Diminishing returns beyond this for a bounded project
Concurrency behavior	At least one dedicated race-condition test	Directly proves NFR-1, the project's core differentiator

17 Error Handling & Validation

17.1 Global Exception Handler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler({StockConflictException.class})
    public ResponseEntity<ErrorResponse> handleStockConflict(StockConflictException ex, HttpServletRequest req) {
        return build(HttpStatus.CONFLICT, "STOCK_CONFLICT", ex.getMessage(), req);
    }

    @ExceptionHandler({IllegalStateException.class})
    public ResponseEntity<ErrorResponse> handleIllegalTransition(IllegalStateException ex, HttpServletRequest req) {
        return build(HttpStatus.CONFLICT, "ILLEGAL_STATE_TRANSITION", ex.getMessage(), req);
    }

    @ExceptionHandler({MethodArgumentNotValidException.class})
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex, HttpServletRequest req) {
        String message = ex.getBindingResult().getFieldErrors().stream()
            .map(f -> f.getField() + ": " + f.getDefaultMessage())
            .collect(Collectors.joining("; "));
        return build(HttpStatus.BAD_REQUEST, "VALIDATION_ERROR", message, req);
    }

    @ExceptionHandler({AccessDeniedException.class})
    public ResponseEntity<ErrorResponse> handleAccessDenied(AccessDeniedException ex, HttpServletRequest req) {
        return build(HttpStatus.FORBIDDEN, "ACCESS_DENIED", "Insufficient permissions", req);
    }

    private ResponseEntity<ErrorResponse> build(HttpStatus status, String code, String message, HttpServletRequest req) {
        var body = new ErrorResponse(Instant.now(), status.value(), code, message, req.getRequestURI());
        return ResponseEntity.status(status).body(body);
    }
}
```

17.2 Bean Validation on Input DTOs

```
public record OrderLineRequest(
    @NotNull UUID productId,
    @Min(1) @Max(10_000) int quantity
) {}

public record OrderRequest(
    @NotNull UUID customerId,
    @NotEmpty @Size(max = 50) @Valid List<OrderLineRequest> lines
) {}
```

Rejecting bad input at the DTO boundary (rather than deep inside the service layer) keeps invalid data out of the routing algorithm entirely — the algorithm can safely assume its inputs are well-formed.

18 Scheduled Jobs & Twilio Reorder Alerts

```
@Component
public class ReorderAlertJob {

    private final StockItemRepository stockItemRepository;
    private final TwilioSmsClient smsClient;

    @Scheduled(cron = "0 0 6 * * *") // 6 AM daily
    public void checkReorderThresholds() {
        List<StockItem> lowStock = stockItemRepository.findBelowThreshold();
        Map<Warehouse, List<StockItem>> byWarehouse = lowStock.stream()
            .collect(Collectors.groupingBy(StockItem::getWarehouse));

        byWarehouse.forEach((warehouse, items) -> {
            String summary = items.stream()
                .map(i -> i.getProduct().getSku() + ": " + i.getQuantity() + " left")
                .collect(Collectors.joining(", "));
            warehouse.getManagers().forEach(manager ->
                smsClient.send(manager.getPhone(),
                    "STRIDE alert - " + warehouse.getName() + " low stock: " + summary));
        });
    }
}

@Component
public class TwilioSmsClient {
    @Value("${twilio.from-number}") private String fromNumber;

    public void send(String toNumber, String body) {
        try {
            Message.creator(new PhoneNumber(toNumber), new PhoneNumber(fromNumber), body).create();
        } catch (Exception e) {
            log.warn("Twilio SMS failed for {}: {}", toNumber, e.getMessage()); // non-fatal - never block
        }
    }
}
```

Note the try/catch around the Twilio call — a third-party network failure must never take down the scheduled job for every other warehouse, and it must never propagate into the order-placement critical path (this is why it lives in the scheduler, not inside `OrderService`).

19 DevOps & Deployment

19.1 docker-compose.yml

```
version: "3.9"
services:
  postgres:
    image: postgres:15
    environment:
      POSTGRES_DB: smartroute
      POSTGRES_USER: smartroute
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U smartroute"]
      interval: 5s
      timeout: 5s
      retries: 5

  app:
    build: .
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/smartroute
      SPRING_DATASOURCE_USERNAME: smartroute
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
      JWT_SECRET: ${JWT_SECRET}
      TWILIO_ACCOUNT_SID: ${TWILIO_ACCOUNT_SID}
      TWILIO_AUTH_TOKEN: ${TWILIO_AUTH_TOKEN}
    ports: ["8080:8080"]

volumes:
  pgdata:
```

19.2 Dockerfile (multi-stage build)

```
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package -DskipTests

FROM eclipse-temurin:17-jre-alpine
```

```
WORKDIR /app
COPY --from=build /app/target/smartroute-*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

19.3 Environment Variables (.env.example)

```
DB_PASSWORD=changeme
JWT_SECRET=changeme-use-a-long-random-string
TWILIO_ACCOUNT_SID=your_sid
TWILIO_AUTH_TOKEN=your_token
```

Running the entire system is a single command: `docker compose up --build`, satisfying NFR-6.

20 Optional Demo Dashboard

A thin React page (not the project's focus — budget at most 2-3 days) that:

- Lists open orders with status badges.
- On selecting an order, shows a Leaflet.js map (already familiar from the NavIC internship work) plotting the customer location and each allocated warehouse, with a line to each, and the score breakdown shown as a tooltip.
- Provides a simple form to place a test order.

This is valuable specifically because it makes the routing algorithm's output *visible* rather than something read out of a JSON response during a demo — a live map showing an order splitting across two warehouses is a much stronger 90-second demo than scrolling through API responses.

21 Three-Week Build Plan (Day by Day)

21.1 Week 1 — Foundations

Day	Focus
1	Project scaffold (Spring Initializr), package structure, Flyway migration V1, Docker Compose up and running
2	Entities + repositories for Warehouse, Product, StockItem, Customer
3	User entity, Spring Security config, JWT issuance (login/register), password hashing
4	JwtFilter, @PreAuthorize role checks, warehouse-manager scoping guard
5	CRUD endpoints: Warehouse, Product, Stock (with DTOs + validation), Swagger wired up
6-7	Buffer / catch-up; write unit tests for everything built so far

21.2 Week 2 — The Core Differentiator

Day	Focus
8	StockSnapshot, LineRequest, AllocationPlan records; RoutingStrategy interface
9	DistanceCostStrategy — Phase 1 (candidate filtering) + Phase 2 (greedy scoring) with unit tests
10	ConsolidationPlanner — bitmask DP, with unit tests proving consolidation beats naive line-by-line in the intended case
11	OrderService orchestration, StockAllocationExecutor, @Version + @Retryable wiring
12	Order state machine (OrderState implementations), transition-guarded endpoints
13	POST /orders end-to-end wired, integration test for the happy path
14	Concurrency test (Section 16.4) — this is the day to get this working and demo-ready

21.3 Week 3 — Polish, Proof, Presentation

Day	Focus
15	Global exception handler, consistent error schema, remaining validation
16	Reorder scheduled job + Twilio integration
17	Testcontainers integration suite completed; coverage check against Section 16.5 targets
18	Dockerfile + docker-compose finalized, fresh-clone docker compose up verification
19	README (architecture diagram, “why greedy not ILP” explanation, setup instructions), Postman/OpenAPI collection exported
20	Optional: React demo dashboard with Leaflet map
21	Final rehearsal: run through the viva Q&A (Section 23), rehearse the live concurrency-test demo

22 Evaluation Criteria Mapping

Typical evaluation criterion	Where STRIDE demonstrates it
Problem understanding / originality	Section 2–3: explicit differentiation from tutorial-style inventory projects
Requirements & design discipline	Section 4 (FR/NFR), Section 5 (layered architecture with a justified deviation)
Data modeling	Section 7 (entities), Section 14 (schema with constraints, not just types)
Algorithms & complexity analysis	Section 10 (full derivation, complexity table, rejected-alternative justification)
OOP principles	Section 11 (four named patterns, each tied to a specific problem solved)
Concurrency / systems thinking	Section 12 (the race condition, the fix, the retry-scope detail most people miss)
Security	Section 13 (JWT, RBAC, method-level enforcement, secret handling)
Testing rigor	Section 16 (unit + integration + a dedicated concurrency test — the strongest evidence in the whole project)
Software engineering practice	Flyway migrations (not <code>ddl-auto=update</code>), global exception handling, DTO validation, OpenAPI docs
DevOps / deployability	Section 19 (single-command startup)
Communication	Section 23 (viva prep) — being able to explain <i>why</i> , not just <i>what</i>

23 Viva / Evaluation Q&A Preparation

Q1: Why didn't you just use an existing optimization library for the routing?

A: An exact solver (ILP/LP) gives a global optimum but has no bounded runtime guarantee, which is unacceptable for a synchronous order-placement API. The greedy + bounded bitmask DP approach trades a small amount of optimality for a bounded, fast, and *explainable* result — and the candidate pre-filter keeps the DP's exponential term small regardless of total system size. (Section 10.2, 10.6)

Q2: What happens if two orders try to buy the last unit at the same time?

A: Optimistic locking via a `@Version` column on `StockItem` — walk through Section 12.1–12.3, and if possible, run the concurrency test (Section 16.4) live.

Q3: Why optimistic locking instead of just locking the row? A: Pessimistic locks serialize every writer to a given stock row even when most requests don't actually conflict; optimistic locking only pays a retry cost when a real conflict happens, which better matches the expected access pattern. (Section 12.2)

Q4: What's the actual bug in retrying just the database save, and why does your solution avoid it? A: The in-memory allocation plan was computed against a now-stale stock snapshot; retrying only the save re-attempts writing stale data. The retry boundary must cover re-fetching stock and re-running the (pure) routing algorithm, not just the persistence call. (Section 12.3)

Q5: Why is the routing service not a Spring @Service with repository access? A: Keeping it a pure function (no Spring, no DB) makes it unit-testable in milliseconds without a database or application context, and makes it safe to call repeatedly under retry without side effects accumulating. (Section 5.3)

Q6: Walk me through what happens when I place an order that no single warehouse can fully cover. A: Candidate filtering → greedy per-line allocation across multiple warehouses until either the line is covered or candidates are exhausted → any uncovered remainder becomes `backorderedQuantity`, and the order status becomes `PARTIALLY_ALLOCATED` rather than being rejected outright. (Sections 8.1, 10.4)

Q7: What's the time complexity of your consolidation step, and how do you keep it from blowing up? A: $O(2^k \cdot L)$ where k is the candidate warehouse count and L the number of order lines; k is capped at 15 by the Phase 1 pre-filter (geography + stock availability), which keeps 2^k trivially small regardless of how many total warehouses exist in the system. (Section 10.5–10.6)

Q8: Why did you choose PostgreSQL over MongoDB, given you've used MongoDB before? A: The domain is inherently relational with strong consistency requirements across orders, lines, allocations, and stock — exactly the case optimistic locking and foreign-key constraints are built for. MongoDB's eventually-consistent defaults work against the concurrency guarantees the project is specifically built to demonstrate. (Section 6)

Q9: How would you extend this to real delivery-route optimization, not just warehouse selection? A: That's a distinct, harder problem (vehicle routing), explicitly out of scope (Section 2.3); the natural extension point is swapping in a new

RoutingStrategy implementation without touching orchestration code, exactly because of the Strategy pattern boundary already in place. (Section 11.1, Section 24)

Q10: What would you do differently with more time? A: See Section 24 — demand-driven reorder thresholds using order velocity, an ILP-backed strategy for offline batch re-optimization runs (where the bounded-runtime constraint doesn't apply), and expiry-aware FEFO allocation for a perishable-goods vertical.

24 Future Work & Extensions

- **Demand-driven reordering:** replace the static `reorderThreshold` with a rolling average of recent order velocity per product per warehouse, auto-generating purchase suggestions rather than a fixed trigger.
- **Expiry-aware (FEFO) allocation:** for a pharma/cold-chain vertical, extend `StockItem.batchExpiry` into the scoring function so near-expiry batches are preferentially allocated first.
- **Offline ILP re-optimization:** run a full ILP solve as a nightly batch job (where the bounded-runtime constraint of a synchronous API doesn't apply) to periodically re-balance stock across warehouses, using the greedy/DP approach only for real-time order placement.
- **Reverse logistics:** model returns and restocking decisions as a second, related state machine.
- **Event-driven notifications:** replace the direct Twilio call in the scheduler with a message queue (e.g., RabbitMQ) publish, decoupling notification delivery from the scheduled job's execution.
- **Multi-tenant support:** if extended toward a SaaS product, add an organization boundary above `Warehouse/User`.

25 Appendix

25.1 Full Entity List

Warehouse, Product, StockItem, Customer, Order, OrderLine, OrderAllocation, User (Section 7).

25.2 Glossary

Available-to-promise — physical quantity minus reserved quantity; the number actually offerable to a new order. **Backorder** — the portion of a requested quantity that could not be allocated from any candidate warehouse. **Candidate warehouse** — a warehouse that passed the Phase 1 filter (active, in-stock, within radius) for a given order line. **Consolidation** — preferring fewer distinct warehouses per order, even at a small per-line score cost, to reduce shipment fragmentation. **Optimistic locking** — a concurrency control strategy that detects conflicting writes after the fact via a version check, rather than preventing them up front with a lock. **Shortfall penalty** — the scoring component that penalizes a warehouse for being unable to fully cover a line's requested quantity alone.

25.3 Maven Dependencies (pom.xml excerpt)

```
<dependencies>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-web</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-jpa</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-security</artifactId></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-validation</artifactId></dependency>
  <dependency><groupId>org.springframework.retry</groupId><artifactId>spring-retry</artifactId></dependency>
  <dependency><groupId>org.postgresql</groupId><artifactId>postgresql</artifactId><scope>runtime</scope></dependency>
  <dependency><groupId>org.flywaydb</groupId><artifactId>flyway-core</artifactId></dependency>
  <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>0.12.5</version></dependency>
  <dependency><groupId>com.twilio.sdk</groupId><artifactId>twilio</artifactId><version>10.5.0</version></dependency>
  <dependency><groupId>org.springdoc</groupId><artifactId>springdoc-openapi-starter-webmvc-ui</artifactId></dependency>
  <dependency><groupId>org.testcontainers</groupId><artifactId>postgresql</artifactId><scope>test</scope></dependency>
  <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-test</artifactId></dependency>
</dependencies>
```

25.4 Closing Note

Every section of this document is meant to be a working reference while coding, not just something submitted and forgotten. When implementation reveals a detail this specification didn't anticipate, the update belongs in the document too — a report that evolves alongside working code, and can show that evolution, is itself evidence of real engineering practice rather than a specification written after the fact to match already-finished code.